

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Machine Learning (CO3117)

Multilabel Toxic Comment Classification

Assignment Report

Instructor: Truong Vinh Lan

Class: CC01

Students: Pham Nhat Nam - 2352785
Tran Thuy Thuy Truc - 2353259
Nguyen Vu Long - 2352696
Le Phuoc Vu - 2353341
Nguyen Hoang Quoc - 2353027

Ho Chi Minh City, May 2026



Member List & Workload

Fullname	Student ID	Responsibilities	Evaluation
Pham Nhat Nam	2352785		100%
Tran Thuy Thuy Truc	2353259		100%
Nguyen Vu Long	2352696		100%
Le Phuoc Vu	2353341		100%
Nguyen Hoang Quoc	2353027		100%

Team Evaluation Summary:

All team members actively participated and completed their assigned workloads strictly on schedule, resulting in a 100% completion rate across the board. The group maintained a highly collaborative environment throughout the project. Every member demonstrated a strong willingness to learn, adapt to constructive feedback, and continuously synchronize their respective sections to ensure a cohesive, unified, and high-quality final report.



Contents

Member List & Workload	1
1 Exploratory Data Analysis	4
1.1 Dataset	4
1.1.1 Overview	4
1.1.2 Missing Values Check	4
1.1.3 Duplicate Check	5
1.2 Problem Characteristics	5
1.3 Class Imbalance	5
1.3.1 Label Overview	5
1.3.2 Class Distribution Visualization	6
1.3.3 Imbalance Ratio	6
1.3.4 Implications for Modeling	7
1.4 Multi-label Analysis	8
1.4.1 Number of Labels per Comment	8
1.4.2 Label Co-occurrence	8
1.4.3 Label Correlation	9
1.5 Text Statistics	11
1.5.1 Descriptive Statistics	11
1.5.2 Length Distribution Visualization	11
2 Data Preprocessing	13
2.1 Text Normalization Pipeline	13
2.2 Light Text Cleaning Pipeline	13
2.3 Pipeline Usage	13
2.4 Preprocessing Rationale	14
3 Traditional Machine Learning Pipeline	15
3.1 Dataset Loading and Label Definition	15
3.2 Text Normalisation	16
3.3 TF-IDF Feature Extraction	17
3.3.1 Idea of TF-IDF	17
3.3.2 N-gram Representation	18
3.3.3 Illustrative TF-IDF Example	19
3.3.4 Why choosing TF-IDF for the Traditional Pipeline	19
3.4 Multilabel-Stratified Validation Design	20
3.5 Hyperparameter Optimisation with Optuna	20
3.6 Per-label Threshold Calibration	22



3.7	Holdout Validation	22
3.8	Final Training and Test Prediction	23
3.9	Comprehensive Test Evaluation	23
3.10	Traditional Pipeline Evaluation Results	24
3.10.1	Aggregate Evaluation Results	25
3.10.2	Per-Label Performance	26
3.10.3	Receiver Operating Characteristic (ROC) and Precision-Recall Analysis	27
3.11	Summary of Traditional Pipeline	29
4	Deep Learning Pipeline	30
4.1	Feature Extraction	30
4.2	Model Training	31
4.2.1	Model Architecture	31
4.2.2	Loss Function and Imbalanced Data Handling	31
4.2.3	Optimization Strategy	32
4.3	Model Evaluation	32
4.3.1	Thresholding and Metrics Selection	32
4.3.2	Standardized Reporting Architecture	33
4.4	Deep Learning Pipeline Evaluation Results	33
4.4.1	Aggregate Evaluation Results	34
4.4.2	Per-Label Performance	34
4.4.3	Receiver Operating Characteristic (ROC) and Precision-Recall Analysis	34
4.4.4	Summary of Deep Learning Pipeline	36

1 Exploratory Data Analysis

1.1 Dataset

1.1.1 Overview

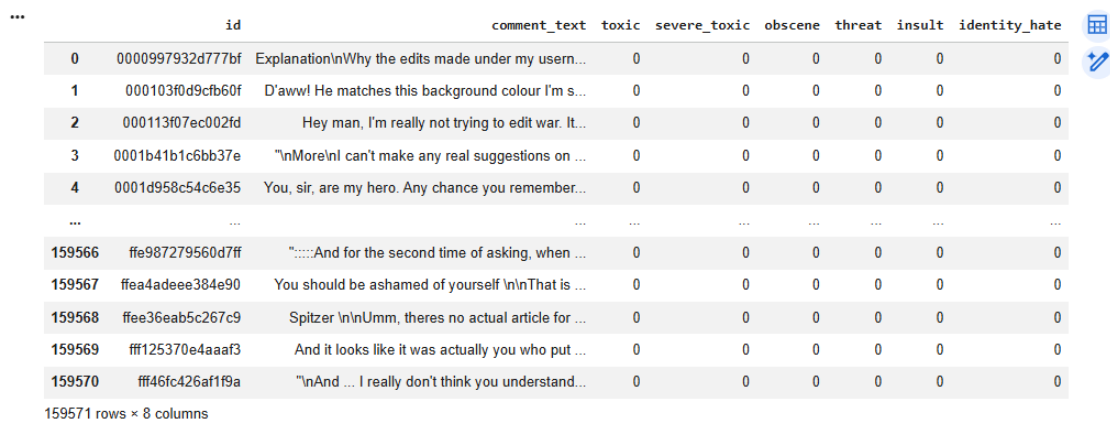
The **Jigsaw Toxic Comment Classification** dataset is a collection of approximately **159,571 comments** from Wikipedia's discussion forum. Each comment is labeled by human annotators with six toxicity categories:

Label	Meaning	Describe
toxic	General toxicity	Rude and disrespectful comments
severe_toxic	Extremely toxic	The most serious case
obscene	Obscene	Vulgar and offensive language
threat	Threat	Threatening the safety of others
insult	Offense	To belittle or insult others
identity_hate	Identity hatred	Hatred based on race, gender, etc

Table 1: Labels in the Jigsaw Toxic Comment dataset

Basic characteristics:

- 159,571 samples and 8 columns (1 id, 1 text, 6 labels).
- No missing values; and no duplicate comments.



	id	comment_text	toxic	severe_toxic	obscene	threat	insult	identity_hate
0	0000997932d777bf	Explanation\nWhy the edits made under my usern...	0	0	0	0	0	0
1	000103f0d9cfb60f	D'aww! He matches this background colour I'm s...	0	0	0	0	0	0
2	000113f07ec002fd	Hey man, I'm really not trying to edit war. It...	0	0	0	0	0	0
3	0001b41b1c6bb37e	"\nMore\nI can't make any real suggestions on ...	0	0	0	0	0	0
4	0001d958c54c6e35	You, sir, are my hero. Any chance you remember...	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...
159566	ffe987279560d7ff	"::::And for the second time of asking, when ...	0	0	0	0	0	0
159567	ffea4adeee384e90	You should be ashamed of yourself \n\nThat is ...	0	0	0	0	0	0
159568	ffee36eab5c267c9	Spitzer \n\nUmm, theres no actual article for ...	0	0	0	0	0	0
159569	fff125370e4aaaf3	And it looks like it was actually you who put ...	0	0	0	0	0	0
159570	fff46fc426af1f9a	"\nAnd ... I really don't think you understand...	0	0	0	0	0	0

159571 rows x 8 columns

Figure 1: train_df output in Colab

1.1.2 Missing Values Check

Number of Null values: 0

comment_text : 0

toxic : 0

severe_toxic : 0

obscene : 0
threat : 0
insult : 0
identity_hate : 0

Conclusion: The dataset contains **no missing values**. Jigsaw is a well-curated dataset where all columns are complete. Therefore, no imputation techniques are required.

1.1.3 Duplicate Check

A check for duplicate content was performed using the `comment_text` attribute as the unique identifier using `duplicated()`.

Number of duplicate comments: 0

Conclusion: The dataset contains no duplicate comments. This ensures that the model will not be biased by repeated samples and simplifies the train-test splitting process as no deduplication is required.

1.2 Problem Characteristics

This task is a **multi-label classification** problem, where each comment can belong to multiple categories simultaneously (e.g., *toxic*, *obscene*, *insult*). The labels are **non-mutually exclusive**. This directly affects:

- **Output Layer:** Use 6 independent sigmoid units instead of softmax
- **Loss Function:** Binary Cross-Entropy (BCE) applied per label
- **Evaluation Metrics:** Requires appropriate metrics for multi-label (ROC-AUC macro, F1-macro)

1.3 Class Imbalance

1.3.1 Label Overview

The dataset is highly imbalanced: The 90/10 ratio between clean and toxic is a clear sign of severe class imbalance. In particular, **threat** (0.30%) and **identity_hate** (0.88%) are extremely rare labels.

Overall distribution:

- Comments with at least one toxic label: **16,225** (10.2%)
- Clean comments: **143,346** (89.8%)

Label	Count	Percentage
toxic	15,294	9.58%
severe_toxic	1,595	1.00%
obscene	8,449	5.29%
threat	478	0.30%
insult	7,877	4.94%
identity_hate	1,405	0.88%

Table 2: Label distribution

Implication: The dataset exhibits severe class imbalance. Therefore, **weighted loss functions** and **AUC-based evaluation metrics** are required, as accuracy alone would be misleading.

1.3.2 Class Distribution Visualization

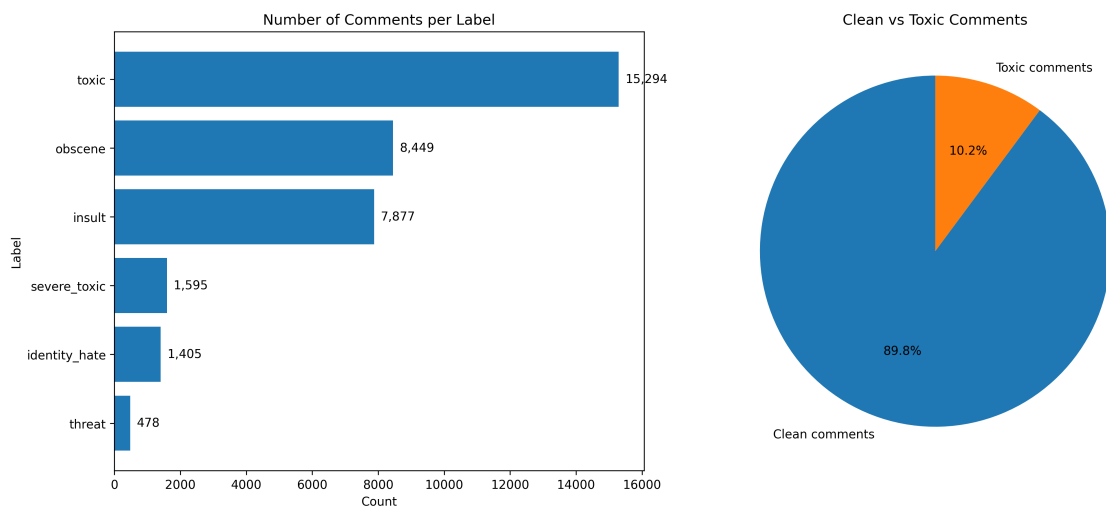


Figure 2: Class distribution and Clean vs Toxic split

Figure 2 shows two important aspects:

- **Left:** The horizontal bar chart displays the number of comments for each label. **toxic** leads with 15,294 samples, while **threat** has only 478 samples - a difference of more than 32x.
- **Right:** The pie chart shows that the majority of the dataset (89.8%) consists of clean comments. Only 10.2% have at least one toxic label.

1.3.3 Imbalance Ratio

To quantify the severity of class imbalance, we compute the imbalance ratio (IR) using the most frequent toxic label, **toxic**, as the reference class:

$$IR(label) = \frac{\text{count of majority label}}{\text{count of current label}}$$

Label	Count	Imbalance Ratio
toxic	15,294	1.0x (majority)
obscene	8,449	1.8x
insult	7,877	1.9x
severe_toxic	1,595	9.6x
identity_hate	1,405	10.9x
threat	478	32.0x

Table 3: Imbalance ratio across toxicity labels

The imbalance ratio shows that the rarest label, **threat**, is approximately **32 times** less frequent than the majority toxic label. Similarly, **identity_hate** and **severe_toxic** are also highly underrepresented.

1.3.4 Implications for Modeling

This imbalance has several important consequences for model development:

1. **Accuracy is not appropriate.** A naive model that always predicts “not threat” would achieve very high accuracy for the **threat** label, but would fail to detect actual threat comments.
2. **Weighted loss function is required.** We use class weights in the binary cross-entropy loss so that mistakes on rare labels receive higher penalties:

$$weight(c) = \frac{total_samples}{count(c)}$$

For the Deep-learning model, we use `BCEWithLogitsLoss` with:

$$pos_weight(c) = \sqrt{\frac{neg_count(c)}{pos_count(c)}}$$

This *sqr-smoothed* variant prevents extreme weights on severely rare labels (e.g., **threat** at 0.3%) from destabilizing training.

3. **AUC-based metrics are more suitable.** ROC-AUC macro is used because it evaluates the ranking ability of the model, independent of threshold, and handles imbalanced data well.
4. **Oversampling may be considered.** Oversampling rare labels such as **threat** and **identity_hate** can be considered, although weighted loss is often sufficient for this dataset.

1.4 Multi-label Analysis

1.4.1 Number of Labels per Comment

Since the task is multi-label, each comment may contain zero, one, or multiple toxicity labels. Table 4 shows the distribution of the number of active labels per comment.

Number of Labels	Count	Percentage
0	143,346	89.83%
1	6,360	3.99%
2	3,480	2.18%
3	4,209	2.64%
4	1,760	1.10%
5	385	0.24%
6	31	0.02%

Table 4: Distribution of number of labels per comment

Analysis:

- The majority of comments (89.83%) have no labels (clean).
- Among toxic comments, a notable proportion has 2-3 labels simultaneously (4.82%).
- There are 31 comments labeled with all 6 labels - these represent extremely toxic cases.

1.4.2 Label Co-occurrence

To understand which labels tend to appear together, we compute a label co-occurrence matrix. This matrix counts how often two toxicity labels are assigned to the same comment.

Most Common Pair	% Overlap
toxic + severe_toxic	100%
toxic + obscene	94%
toxic + insult	93%
obscene + insult	78%
toxic + identity_hate	93%

Table 5: Most common co-occurring label pairs by percentage overlap

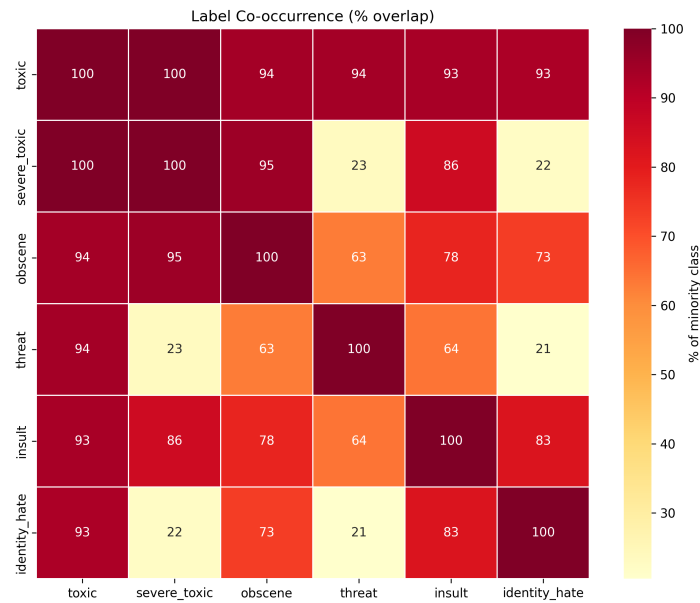


Figure 3: Label co-occurrence matrix

Figure 3 shows that `toxic`, `severe_toxic`, `obscene`, and `insult` frequently co-occur. This suggests that many offensive comments contain both vulgar language and personal attacks.

Key observations:

- `toxic` acts as an “umbrella” label, since most toxic-related categories strongly co-occur with it.
- `obscene` + `insult` is the most common pair outside of `toxic`. This suggests that profanity often appears together with personal attacks.
- `severe_toxic` always co-occurs with `toxic` (100%). This is expected because extremely toxic comments are also generally toxic.
- `threat` is relatively independent, with only 23% overlap with `severe_toxic`. This indicates that threats do not necessarily involve profanity or extreme abusive language.

Conclusion: The model should use independent sigmoid outputs rather than softmax, because multiple labels can be active simultaneously. Shared representations are also useful because related labels frequently co-occur.

1.4.3 Label Correlation

In addition to co-occurrence, label correlations provide insights into the strength of the relationship between two labels. We compute the Pearson correlation matrix for the

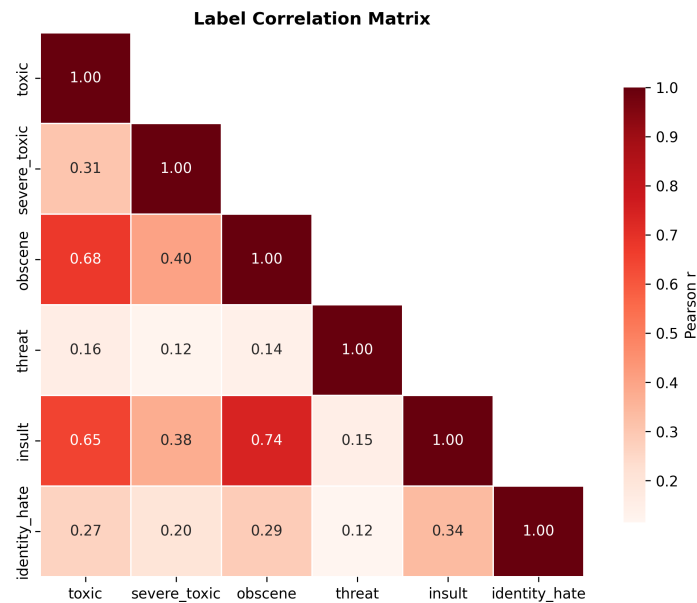


Figure 4: Label correlation matrix (Pearson r)

six labels to examine how labels are correlated with each other. Figure 4 displays the Pearson correlation matrix.

Strong correlations ($r > 0.4$):

- **obscene - insult** ($r = 0.74$) – Very strong correlation. Obscene comments often co-occur with insults.
- **toxic - obscene** ($r = 0.68$) – Obscene language is almost always labeled as toxic.
- **toxic - insult** ($r = 0.65$) – Insulting comments are also usually toxic.

Weaker correlations:

- **threat - toxic** ($r = 0.16$) – Threats are relatively independent.
- **threat - obscene** ($r = 0.14$) – Threats do not necessarily use obscene language.
- **threat - identity_hate** ($r = 0.12$) – The two types of toxicity are different.

Implications for Modeling:

- **threat** is the hardest label to learn due to its rarity and weak correlation with other labels.
- Higher class weight is needed for **threat** during training.
- Transformer models have an advantage in learning implicit label correlations through shared representations - when the model detects strong “obscene” signals, it automatically boosts probabilities for “insult” and “toxic”.

1.5 Text Statistics

This subsection presents an overview of the text statistics, including **descriptive statistics** for comment length (in characters and words), as well as **token coverage analysis** based on the maximum sequence length.

1.5.1 Descriptive Statistics

The table below summarizes the **mean, median, standard deviation**, and the **95th and 99th percentiles** for both the character count and word count of the comments.

Group	Metric	Characters	Words
All	Mean	394	67
	Median	205	36
	Std	591	99
	P95	1,355	230
	P99	3,444	567
Clean	Mean	404	69
	Median	216	38
Toxic	Mean	303	53
	Median	128	23

Table 6: Descriptive Statistics of Comments (Characters and Words)

Analysis:

- Toxic comments tend to be shorter than clean comments. This is reasonable - toxic comments are often short, aggressive phrases (“You idiot!”, “Go die”), while clean comments are typically longer, more detailed discussions.
- The 95th and 99th percentiles indicate that some clean comments are very long (up to 2,897 characters), whereas toxic comments have relatively shorter extremes.

1.5.2 Length Distribution Visualization

Figure 5 shows that clean comments have a wider length distribution, while toxic comments are more concentrated in shorter ranges.

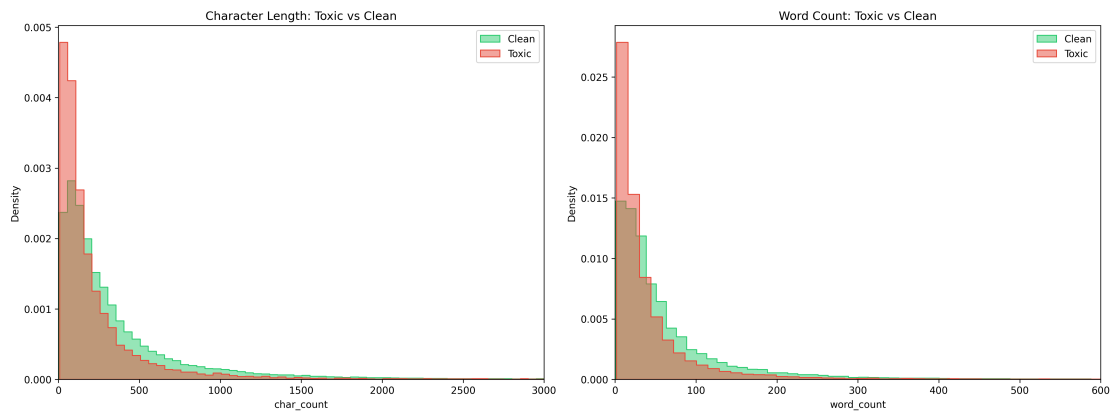


Figure 5: Distribution of character length and word count for clean and toxic comments

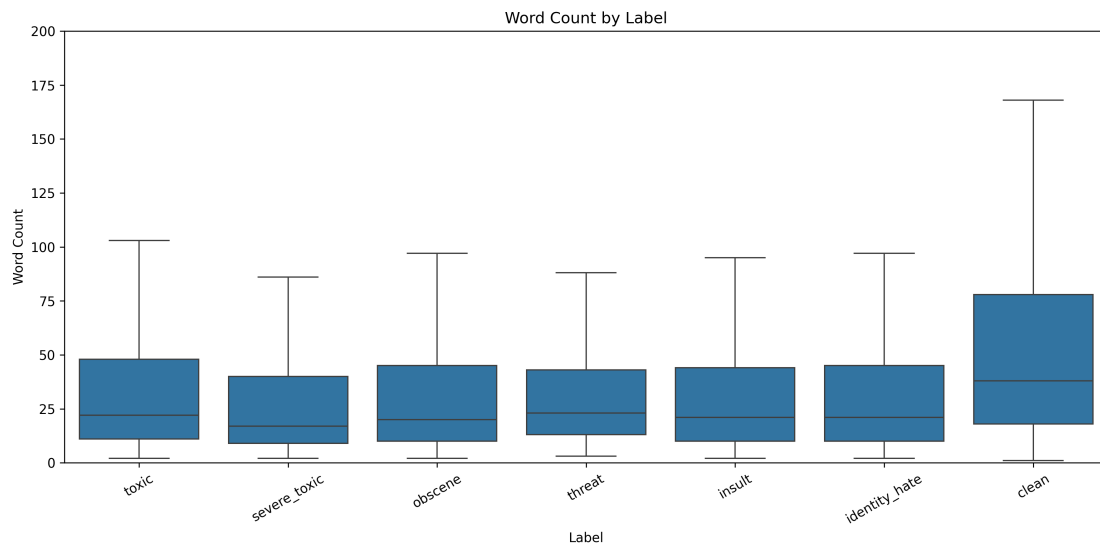


Figure 6: Box plot of word count by toxicity label

Figure 6 compares the word count distribution across the six toxicity labels and clean comments. Clean comments have the widest length distribution and the highest median word count, indicating that non-toxic discussions are generally longer. Among toxic categories, `identity_hate` shows the largest variability, while `threat` comments tend to be shorter and more concentrated in length.

Implication: Toxic comments, especially threat-related comments, are often short and direct. Therefore, extremely long input sequences are not necessary for most toxic samples, supporting the use of a moderate maximum sequence length during model training.

2 Data Preprocessing

The preprocessing stage was designed to reduce noise in the raw comments while preserving information useful for toxicity detection. Two text preprocessing pipelines were used: an aggressive normalization pipeline for traditional machine learning models and a lighter cleaning pipeline for deep learning models.

2.1 Text Normalization Pipeline

The first preprocessing function, `normalize_text()`, applies stronger normalization. This version is mainly suitable for traditional machine learning approaches such as TF-IDF, Logistic Regression, or SVM, where reducing vocabulary size is important.

Step	Description
Lowercasing	Converts all text to lowercase
Unicode normalization	Converts accented characters to ASCII form
Contraction expansion	Expands contractions such as <code>don't</code> to <code>do not</code>
HTML cleaning	Decodes HTML entities and removes HTML tags
URL handling	Replaces URLs with the token <code>URL</code>
IP address removal	Removes IP address patterns
Punctuation removal	Removes punctuation characters
Stopword removal	Removes English stopwords while keeping <code>not</code>
Stemming	Applies Porter stemming to reduce words to root form
Whitespace normalization	Removes extra spaces

Table 7: Steps in the text normalization pipeline

The word `not` was intentionally kept during stopwords removal because negation can change the meaning of a sentence and is important in toxicity detection.

2.2 Light Text Cleaning Pipeline

The second preprocessing function, `clean_text()`, performs lighter cleaning. This pipeline was used for the model training data after splitting into training and testing sets.

Compared with the normalization pipeline, this cleaning method preserves more textual structure, including useful punctuation such as commas, periods, question marks, exclamation marks, apostrophes, and hyphens.

2.3 Pipeline Usage

The normalized text was generated from the full training and test datasets as follows:

```
X_normalize = [normalize_text(text) for text in X_train]
X_test_normalize = [normalize_text(text) for text in X_test]
```

Step	Description
String conversion	Ensures all inputs are treated as text
HTML cleaning	Decodes HTML entities and removes HTML tags
URL handling	Replaces URLs with the token <code>URL</code>
IP address removal	Removes IP address patterns
Unicode normalization	Converts non-ASCII characters to ASCII form
Contraction expansion	Expands shortened expressions
Lowercasing	Converts text to lowercase
Special character filtering	Keeps letters, digits, spaces, and basic punctuation
Whitespace normalization	Collapses multiple spaces into a single space

Table 8: Steps in the light text cleaning pipeline

After splitting the dataset, the lighter cleaning function was applied to the training and testing text:

```
X_clean = [clean_text(text) for text in X_train]
X_test_clean = [clean_text(text) for text in X_test]
```

2.4 Preprocessing Rationale

The two pipelines serve different purposes. The normalization pipeline is more aggressive and is useful for classical models because stemming, punctuation removal, and stopword removal reduce vocabulary sparsity. The cleaning pipeline is less aggressive and better preserves contextual signals such as negation, punctuation, and sentence structure.

Implication: Since toxicity can depend on context, negation, punctuation, and short aggressive phrases, the final preprocessing strategy avoids removing all linguistic cues unless required by the specific model type.

3 Traditional Machine Learning Pipeline

The Traditional Machine Learning Pipeline is implemented as a classical baseline for the multilabel toxic comment classification task. In contrast to the Deep Learning Pipeline, which uses dense contextual sentence embeddings, this pipeline represents comments using sparse lexical statistics and trains a linear discriminative classifier. Its main objective is to provide an interpretable, computationally efficient, and reproducible benchmark for evaluating whether the additional representational complexity of the deep learning approach produces a substantial empirical improvement.

The task is formulated as multilabel classification over six non-mutually exclusive toxicity categories:

$$\mathcal{L} = \{toxic, severe_toxic, obscene, threat, insult, identity_hate\}.$$

Because a single comment may belong to multiple categories simultaneously, the pipeline does not use a multiclass softmax formulation. Instead, the problem is decomposed into six independent binary classification tasks using a one-vs-rest strategy.

The implemented workflow consists of the following stages:

1. loading the Jigsaw Toxic Comment Classification dataset;
2. applying an aggressive text normalisation procedure suitable for sparse lexical modelling;
3. extracting TF-IDF features using `TfidfVectorizer`;
4. storing and reloading sparse feature matrices using the `h5sparse` format;
5. training a one-vs-rest logistic regression classifier;
6. tuning the logistic regression regularisation strength using Optuna;
7. calibrating decision thresholds independently for each label;
8. evaluating the final predictions using multilabel classification metrics.

3.1 Dataset Loading and Label Definition

The dataset is downloaded from the HuggingFace-hosted version of the Jigsaw Toxic Comment Classification Challenge. The implementation uses three CSV files:

- `train.csv`, containing the comment text and six binary toxicity labels;
- `test.csv`, containing unseen comments for prediction;
- `test_labels.csv`, containing available ground-truth labels for test-set evaluation.

The six target columns are explicitly defined in the code as:

```
["toxic", "severe_toxic", "obscene", "threat",  
 "insult", "identity_hate"]
```


The exploratory data analysis shows that the dataset is strongly imbalanced: most comments are clean, while labels such as *threat* and *identity_hate* are rare. This motivates the use of class-weighted logistic regression, macro-averaged metrics, and per-label threshold calibration.

3.2 Text Normalisation

Before feature extraction, each comment is processed by the `normalize_text()` function. This function is designed specifically for the Traditional Machine Learning Pipeline, where the classifier relies on surface-level lexical features rather than contextual token representations.

The implemented normalisation procedure applies the following transformations:

1. **Lowercasing:** All characters are converted to lowercase to reduce case-dependent vocabulary duplication.
2. **Unicode normalisation:** Non-ASCII characters are converted to their closest ASCII equivalents using `unidecode`.
3. **Contraction expansion:** English contractions are expanded using the `contractions` library. For example, *don't* is converted into *do not*.
4. **HTML cleaning:** HTML entities are decoded with `html.unescape()`, and HTML tags are removed using regular expression substitution.
5. **URL and IP-address handling:** URLs are replaced with a symbolic URL token, while IP-address-like numeric patterns are removed.
6. **Punctuation removal:** Punctuation characters are removed using Python's `string.punctuation` translation table.
7. **Stopword removal:** English stopwords from NLTK are removed, except for the word *not*, which is retained because negation can change the semantic polarity of a comment.
8. **Stemming:** Remaining tokens are reduced to their morphological stems using the Porter Stemmer.
9. **Whitespace normalisation:** Repeated whitespace is collapsed, and leading or trailing whitespace is removed.

This aggressive normalisation is appropriate for TF-IDF because sparse bag-of-words representations benefit from vocabulary compression. However, it is not used for the deep learning embedding pipeline, where excessive normalisation could remove contextual and syntactic cues required by transformer-based models.

3.3 TF-IDF Feature Extraction

After text normalisation, each comment is transformed into a sparse numerical feature vector using scikit-learn's `TfidfVectorizer`. The purpose of this step is to convert raw text into a mathematical representation that can be used by the logistic regression classifier. Since logistic regression cannot directly process strings, each comment must first be represented as a vector of numerical features.

The implemented vectoriser is configured as:

```
TfidfVectorizer(  
    min_df=1,  
    max_features=10000,  
    ngram_range=(1, 2)  
)
```

This means that the Traditional Machine Learning Pipeline represents each comment using the 10,000 most frequent unigram and bigram features extracted from the normalised training corpus.

3.3.1 Idea of TF-IDF

TF-IDF stands for **Term Frequency–Inverse Document Frequency**. The central idea is to assign high importance to words or phrases that are frequent in a particular comment but not common across the entire dataset. This is useful for toxicity classification because highly discriminative expressions, such as insults, profane terms, or identity-directed phrases, may be strong indicators of toxicity even if they appear in only a subset of comments.

TF-IDF combines two components:

- **Term Frequency (TF):** measures how often a term appears in a specific document. A word that appears many times in one comment receives a higher term-frequency value for that comment.
- **Inverse Document Frequency (IDF):** measures how rare or specific a term is across the entire corpus. Terms that appear in many comments receive lower IDF values, while terms that appear in fewer comments receive higher IDF values.

For a term t in document d from corpus $\mathcal{D}$, the TF-IDF score is:

$$\text{TF-IDF}(t, d, \mathcal{D}) = \text{TF}(t, d) \times \text{IDF}(t, \mathcal{D}). \quad (1)$$

Using scikit-learn's smoothed IDF formulation, the inverse document frequency is:

$$\text{IDF}(t, \mathcal{D}) = \log \left(\frac{1 + |\mathcal{D}|}{1 + |\{d' \in \mathcal{D} : t \in d'\}|} \right) + 1. \quad (2)$$

Here, $|\mathcal{D}|$ is the total number of documents in the training corpus, and $|\{d' \in \mathcal{D} : t \in d'\}|$ is the number of documents containing term t . The smoothing terms prevent division by zero and are consistent with scikit-learn's default implementation.

The intuition is that very common terms receive less importance because they are less useful for distinguishing toxic from non-toxic comments. In contrast, terms or phrases that occur in fewer documents but are strongly associated with toxic language receive larger weights.

3.3.2 N-gram Representation

An **n-gram** is a contiguous sequence of n tokens extracted from text. In this pipeline, `ngram_range=(1,2)` means that the vectoriser extracts:

- **unigrams**, where $n = 1$, corresponding to individual words;
- **bigrams**, where $n = 2$, corresponding to adjacent two-word phrases.

For example, after normalisation, suppose a comment becomes:

`stupid idiot`

With unigram features only, the extracted features would be:

`stupid, idiot`

With `ngram_range=(1,2)`, the extracted features become:

`stupid, idiot, stupid idiot`

Thus, the model can learn not only the importance of individual words such as *stupid* and *idiot*, but also the importance of the phrase *stupid idiot*. This is important because some toxic meanings are better captured by short word combinations than by isolated words alone.

As another example, consider the normalised comment:

`not good person`

The unigram features are:

`not, good, person`

The bigram features are:

`not good, good person`

The bigram *not good* contains information about local word order and negation that would be partly lost if the model only used individual words. Therefore, including bigrams allows the traditional pipeline to capture limited phrase-level context while still using an efficient sparse representation.

3.3.3 Illustrative TF-IDF Example

Consider a small training corpus containing three normalised comments:

```
d1: stupid idiot
d2: stupid comment
d3: thank comment
```

Using unigram and bigram extraction, the possible features include:

```
stupid, idiot, comment, thank,
stupid idiot, stupid comment, thank comment
```

The unigram *stupid* appears in two documents, *d1* and *d2*, so its IDF value is lower because it is less rare:

$$\text{IDF}(\textit{stupid}) = \log \left(\frac{1+3}{1+2} \right) + 1 \approx 1.288. \quad (3)$$

The bigram *stupid idiot* appears only in one document, *d1*, so it receives a higher IDF value:

$$\text{IDF}(\textit{stupid idiot}) = \log \left(\frac{1+3}{1+1} \right) + 1 \approx 1.693. \quad (4)$$

This example shows the main principle of TF-IDF: a feature becomes more important when it is frequent in a specific document but relatively rare across the corpus. In toxicity detection, this is useful because phrase-level abusive patterns may be rare overall but highly informative when they occur.

3.3.4 Why choosing TF-IDF for the Traditional Pipeline

TF-IDF is appropriate for the Traditional Machine Learning Pipeline for several reasons. First, it produces a high-dimensional but sparse representation, which is efficient for linear classifiers such as logistic regression. Second, it provides interpretable lexical features: each dimension corresponds to a specific word or phrase. Third, by using both unigrams and bigrams, the model can capture individual toxic words as well as short abusive expressions.

However, TF-IDF does not understand deep semantic meaning or long-range context. For example, it treats features mainly according to their frequency patterns and does not model sentence meaning in the same way as transformer embeddings. For this reason, TF-IDF serves as a strong classical baseline, while the Deep Learning Pipeline is expected to capture richer contextual information.

3.4 Multilabel-Stratified Validation Design

To obtain reliable validation results under severe multilabel imbalance, the pipeline uses multilabel-stratified splitting from the `iterative-stratification` package. This strategy attempts to preserve the distribution of the six labels across the generated subsets.

The first split divides the TF-IDF training matrix into:

- an 80% fitting subset used for cross-validation and model fitting;
- a 20% temporary subset reserved for threshold tuning and final validation reporting.

The temporary subset is then split equally into:

- a 10% threshold-tuning subset;
- a 10% independent holdout reporting subset.

Thus, the implemented validation structure is:

80% fit, 10% threshold tuning, 10% holdout reporting.

This separation is important because the subset used to choose the decision thresholds is not reused for the final validation report.

3.5 Hyperparameter Optimisation with Optuna

The Traditional Machine Learning Pipeline uses Optuna to optimise the main hyperparameters of the one-vs-rest logistic regression classifier. Instead of manually fixing a single logistic regression configuration, the optimisation procedure searches over several valid combinations of regularisation type, optimisation solver, regularisation strength, and maximum number of iterations.

The primary hyperparameter is the inverse regularisation strength C :

$$C \sim \text{LogUniform}(10^{-3}, 10). \quad (5)$$

A smaller value of C applies stronger regularisation, which can reduce overfitting in the high-dimensional TF-IDF feature space. A larger value of C weakens the regularisation penalty, allowing the classifier to fit the training data more closely. Since the optimal scale of C is not known in advance, it is sampled logarithmically.

In addition to C , the Optuna search space includes multiple regularisation penalties and compatible solvers:

- ℓ_1 **regularisation**, evaluated with solvers that support sparse coefficient selection, such as `liblinear` and `saga`;

- ℓ_2 **regularisation**, evaluated with compatible solvers such as `lbfgs`, `liblinear`, `sag`, and `saga`;

Invalid solver–penalty combinations are excluded from the search space. This is necessary because not every solver in scikit-learn supports every type of regularisation. For example, `lbfgs` supports ℓ_2 regularisation but does not support ℓ_1 , while `saga` supports ℓ_1 , ℓ_2 regularisation.

Each Optuna trial constructs a candidate one-vs-rest logistic regression model using the sampled hyperparameters. The model is evaluated using 5-fold `MultilabelStratifiedKfold` cross-validation on the fitting subset. This cross-validation strategy is used because the dataset is both multilabel and highly imbalanced; therefore, preserving the distribution of all six toxicity labels across folds is more reliable than ordinary random splitting.

For each fold, the classifier is trained on two folds and validated on the remaining fold. The trained model outputs a probability estimate for each label:

$$\hat{p}_c = P(y_c = 1 \mid \mathbf{x}), \quad c \in \mathcal{L}. \quad (6)$$

Because multilabel toxicity classification is highly imbalanced, the validation score is not computed using a fixed threshold of 0.5. Instead, for each label c , a threshold search is performed over a grid:

$$\tau \in \{0.100, 0.105, 0.110, \dots, 0.905\}. \quad (7)$$

For every label, the threshold that maximises the validation F1-score is selected:

$$\tau_c^{*(k)} = \arg \max_{\tau} F_1^{(k)}(c, \tau), \quad (8)$$

where k denotes the validation fold. The score for one fold is then computed as the mean of the best F1-scores across the six toxicity labels:

$$S^{(k)} = \frac{1}{|\mathcal{L}|} \sum_{c \in \mathcal{L}} \max_{\tau} F_1^{(k)}(c, \tau). \quad (9)$$

The objective uses F1-score rather than accuracy because the dataset contains many more negative labels than positive labels. Accuracy would reward a model that predicts the majority negative class, while F1-score penalises both missed toxic comments and incorrect toxic predictions.

The final Optuna objective is the mean score across the five validation folds:

$$J(\theta) = \frac{1}{K} \sum_{k=1}^K S^{(k)}, \quad K = 5, \quad (10)$$

where θ represents the sampled hyperparameter configuration.

The Optuna study maximises this objective. The best configuration is the one that achieves the highest mean cross-validated macro-style F1 score after per-label threshold optimisation. This procedure allows the Traditional Pipeline to compare multiple valid logistic regression configurations rather than relying on a single manually selected solver and penalty.

3.6 Per-label Threshold Calibration

The logistic regression model outputs probabilities rather than direct binary decisions. Therefore, a decision threshold must be selected to convert each predicted probability into a binary label. A single global threshold of 0.5 is often suboptimal for imbalanced multilabel problems because rare labels may have well-ranked but numerically lower predicted probabilities.

The implemented pipeline performs per-label threshold calibration on the separate 10% threshold-tuning subset. For each label c , the threshold is selected by maximising the F1-score over:

$$\tau \in \{0.100, 0.105, 0.110, \dots, 0.945\}.$$

The optimal threshold for label c is:

$$\tau_c^* = \arg \max_{\tau} F_1(c, \tau). \quad (11)$$

The final binary prediction is:

$$\hat{y}_c = \mathbb{I}[\hat{p}_c \geq \tau_c^*], \quad (12)$$

where $\mathbb{I}[\cdot]$ denotes the indicator function.

The calibrated thresholds are saved to:

`best_thresholds.csv`

This file is later reused by the evaluation script, allowing the same calibrated decision rule to be applied consistently.

3.7 Holdout Validation

After threshold calibration, the model is evaluated on the independent 10% holdout reporting subset. This subset is not used during Optuna optimisation or threshold selection, so it provides a less biased estimate of validation performance.

The holdout evaluation reports:

- per-label precision, recall, F1-score, and support using `classification_report`;

- macro precision;
- macro recall;
- macro F1-score.

Macro-averaged metrics are appropriate for this task because they assign equal weight to each toxicity label. This prevents frequent labels such as *toxic* from dominating the evaluation and makes model performance on rare labels such as *threat* more visible.

3.8 Final Training and Test Prediction

After selecting the best regularisation strength and calibrating the per-label thresholds, the final one-vs-rest logistic regression model is retrained on the entire TF-IDF training matrix:

$$X_{\text{train}}^{\text{TFIDF}}, \quad Y_{\text{train}}.$$

The trained model then generates probability predictions for the TF-IDF test matrix:

$$\hat{\mathbf{P}}_{\text{test}} = f(X_{\text{test}}^{\text{TFIDF}}), \quad (13)$$

where each row contains six predicted probabilities, one for each toxicity label.

The calibrated thresholds are applied to obtain binary predictions. The pipeline exports:

- `test_pred_proba.csv`, containing probability predictions;
- `test_pred_binary.csv`, containing thresholded binary predictions.

Maintaining both probability and binary outputs is important because probability outputs are required for threshold-independent metrics such as ROC-AUC, while binary outputs are required for threshold-dependent metrics such as F1-score, subset accuracy, and Hamming Loss.

3.9 Comprehensive Test Evaluation

The evaluation module compares the generated prediction file with `test_labels.csv`. If an `id` column exists in both files, the rows are aligned by ID before metric computation. This ensures that each prediction is matched with the correct ground-truth label vector.

Rows containing the Kaggle sentinel value `-1` are removed before evaluation, because these correspond to unscored test examples. The evaluator supports both probability-valued predictions and already-binary predictions. If probability predictions are supplied, the saved thresholds from `best_thresholds.csv` are applied automatically; otherwise, a global fallback threshold can be used.

The exported metrics include:

- **Subset Accuracy**, the proportion of samples for which all six labels are predicted exactly correctly:

$$\text{SubsetAccuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\hat{\mathbf{y}}^{(i)} = \mathbf{y}^{(i)}]. \quad (14)$$

- **Micro F1-score**, computed globally across all label decisions.
- **Macro F1-score**, computed as the unweighted mean of the six per-label F1-scores.
- **Hamming Loss**, the fraction of incorrectly predicted sample-label pairs:

$$\mathcal{H} = \frac{1}{N|\mathcal{L}|} \sum_{i=1}^N \sum_{c \in \mathcal{L}} \mathbb{I}[\hat{y}_c^{(i)} \neq y_c^{(i)}]. \quad (15)$$

- **Macro ROC-AUC**, computed as the mean of the valid per-label ROC-AUC scores using probability predictions.
- **Per-label Precision, Recall, F1-score, Support, and ROC-AUC**, enabling fine-grained diagnosis of performance across toxicity categories.

The evaluation results are exported to a structured output directory containing:

1. `metrics_summary.json`;
2. `classification_report.json`;
3. `per_label_metrics.csv`;
4. `per_label_prf.png`;
5. `label_support.png`.

This standardised reporting structure makes the Traditional Machine Learning Pipeline directly comparable with the Deep Learning Pipeline, since both models are evaluated using the same core multilabel metrics and output formats.

3.10 Traditional Pipeline Evaluation Results

Following the generation of probability and binary predictions for the official test set, the Traditional Machine Learning Pipeline was evaluated against the ground-truth labels provided in `test_labels.csv`. Samples containing the Kaggle sentinel value of `-1` were excluded prior to metric computation, yielding 63,978 valid aligned test samples. The evaluation utilized calibrated per-label thresholds (stored in `best_thresholds.csv`) rather than a global threshold.

The optimal thresholds applied for final binary predictions were:

$$\begin{aligned} \tau_{toxic} &= 0.795, & \tau_{severe_toxic} &= 0.930, & \tau_{obscene} &= 0.845, \\ \tau_{threat} &= 0.925, & \tau_{insult} &= 0.865, & \tau_{identity_hate} &= 0.945. \end{aligned}$$

3.10.1 Aggregate Evaluation Results

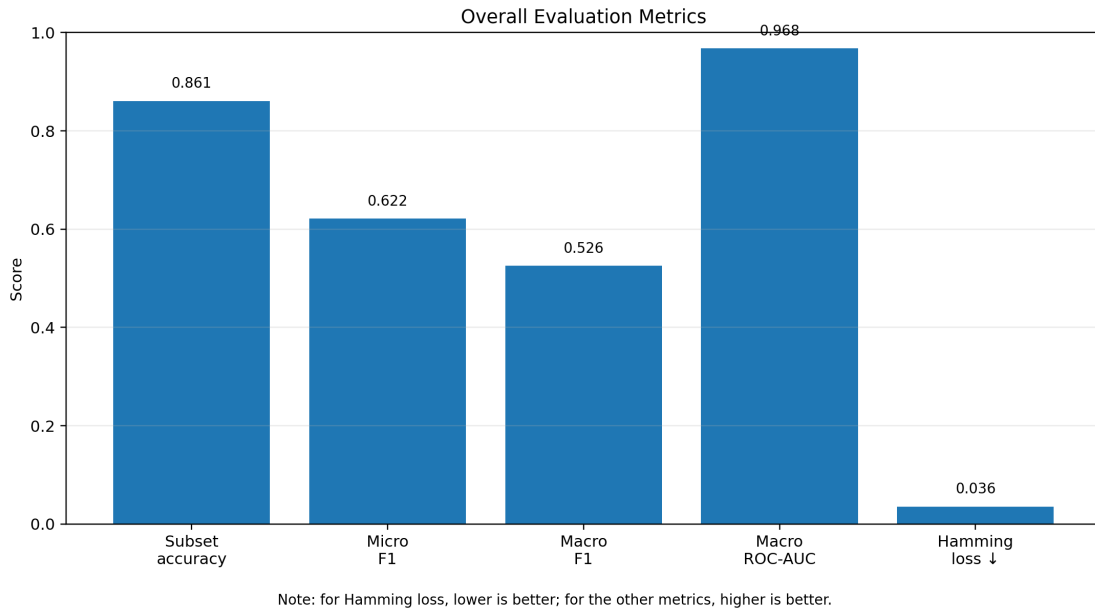


Figure 7: Overall evaluation metrics for the Traditional Machine Learning Pipeline.

As illustrated in Figure 7, the model achieves a high **subset accuracy of 0.861** and a low **Hamming loss of 0.036**. While seemingly strong, these metrics must be interpreted with caution. In multilabel toxicity classification, the feature space is heavily skewed; most comments do not trigger any toxicity labels. Consequently, a model can artificially inflate accuracy and minimize Hamming loss simply by correctly identifying the overwhelming majority of non-toxic (negative) instances.

A more robust evaluation of classification performance is provided by the F1-scores. The model yields a **micro F1-score of 0.622** and a **macro F1-score of 0.526**. The disparity between these two scores highlights the impact of class imbalance: micro F1 aggregates performance globally and is bolstered by the model’s success on more frequent labels (e.g., *toxic*, *obscene*), whereas macro F1 weights each class equally, exposing poorer performance on underrepresented labels.

Notably, the model achieves a stellar **macro ROC-AUC of 0.968**. This indicates exceptional ranking capability across all categories—the pipeline effectively assigns higher probabilities to genuinely toxic comments compared to benign ones. However, the gap between this high ROC-AUC and the lower F1-scores suggests that mapping these well-ranked probabilities to concrete binary outcomes via static thresholds re-

mains a challenge under severe class imbalance.

3.10.2 Per-Label Performance

Conversely, the model struggles with the rarest categories, *threat* and *severe_toxic*, which register the lowest F1-scores of 0.37 and 0.34, respectively. Due to the scarcity of positive examples in these classes, the classifier lacks sufficient evidence to establish robust decision boundaries, making the metrics highly sensitive to individual false positives and false negatives.

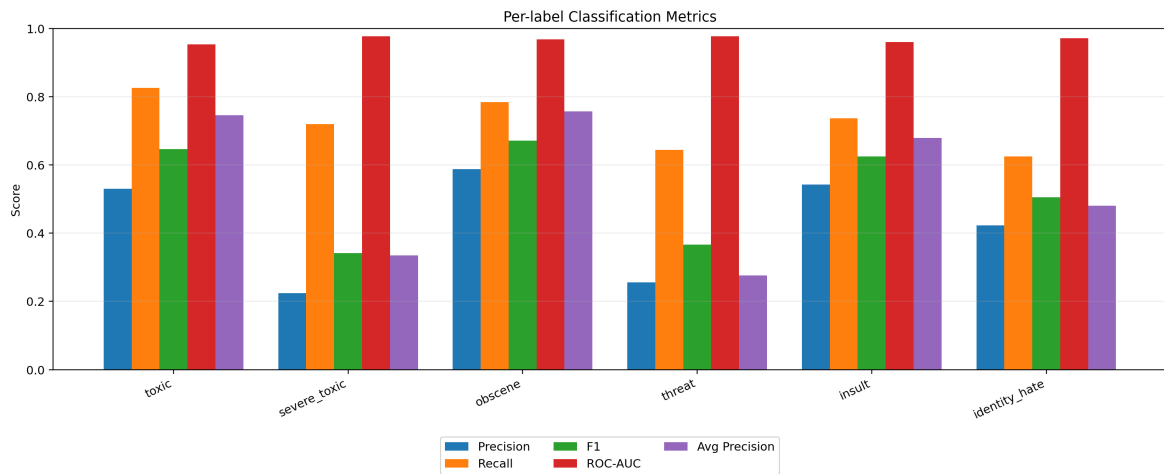


Figure 8: Per-label classification metrics for the six toxicity categories.

A prominent trend across all six labels is that **recall consistently outpaces precision**. For instance, the *toxic* label achieves a recall of 0.83 but a precision of only 0.53, while *severe_toxic* sees a recall of 0.72 against a precision of 0.22. This dynamic demonstrates that the calibrated thresholds are biased toward sensitivity: the model successfully catches the majority of toxic content (high recall) but generates a notable volume of false positives in the process (low precision). In the context of content moderation, this conservative behaviour is often preferred, as the cost of missing severely toxic content generally outweighs the inconvenience of reviewing false flags.

Category	Precision	Recall	F1	ROC-AUC	PR-AUC	Support
toxic	0.53	0.83	0.65	0.9541	0.7457	6090
severe_toxic	0.22	0.72	0.34	0.9772	0.3347	367
obscene	0.59	0.78	0.67	0.9678	0.7569	3691
threat	0.26	0.64	0.37	0.9771	0.2762	211
insult	0.54	0.74	0.62	0.9604	0.6796	3427
identity_hate	0.42	0.62	0.50	0.9716	0.4803	712

Table 9: Per-label classification performance across toxicity categories.

3.10.3 Receiver Operating Characteristic (ROC) and Precision-Recall Analysis

To further contextualize the threshold-dependent metrics discussed above, it is essential to examine the model's threshold-independent performance across all possible classification boundaries. This is best visualized through the Receiver Operating Characteristic (ROC) and Precision-Recall (PR) curves for each label.

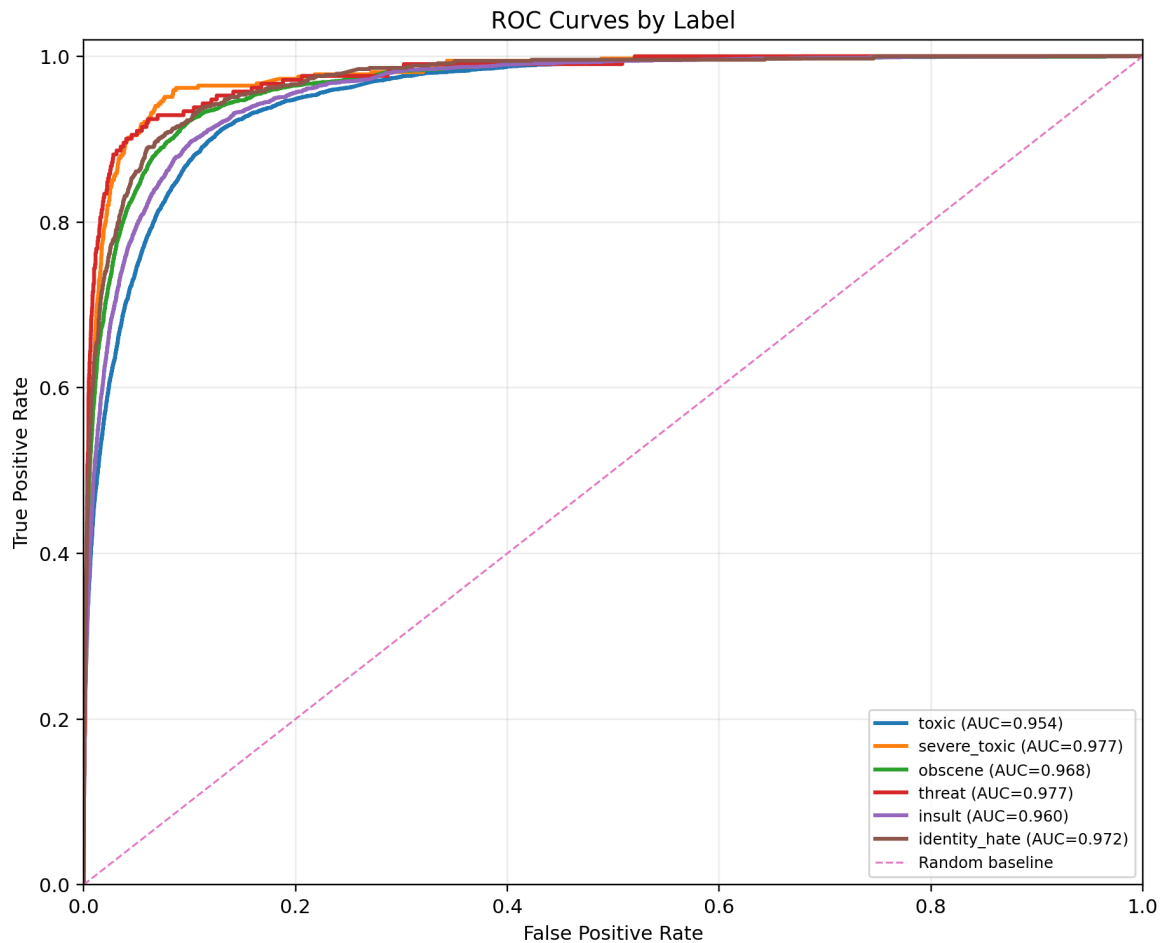


Figure 9: ROC Curves for Toxicity Labels.

As depicted in Figure 9, the model exhibits excellent separability across all categories, with every curve bowing sharply toward the top-left corner. This is reflected in the near-perfect AUC scores, which range from 0.954 for *toxic* to an impressive 0.977 for the rarest classes, *severe_toxic* and *threat*. However, while these plots suggest outstanding performance, ROC curves can present an overly optimistic view in the presence of severe class imbalance. Because the False Positive Rate (the x-axis) is calculated relative to the massive pool of true negatives (benign comments), even a large absolute number of false positives will barely register, keeping the curve artificially elevated.

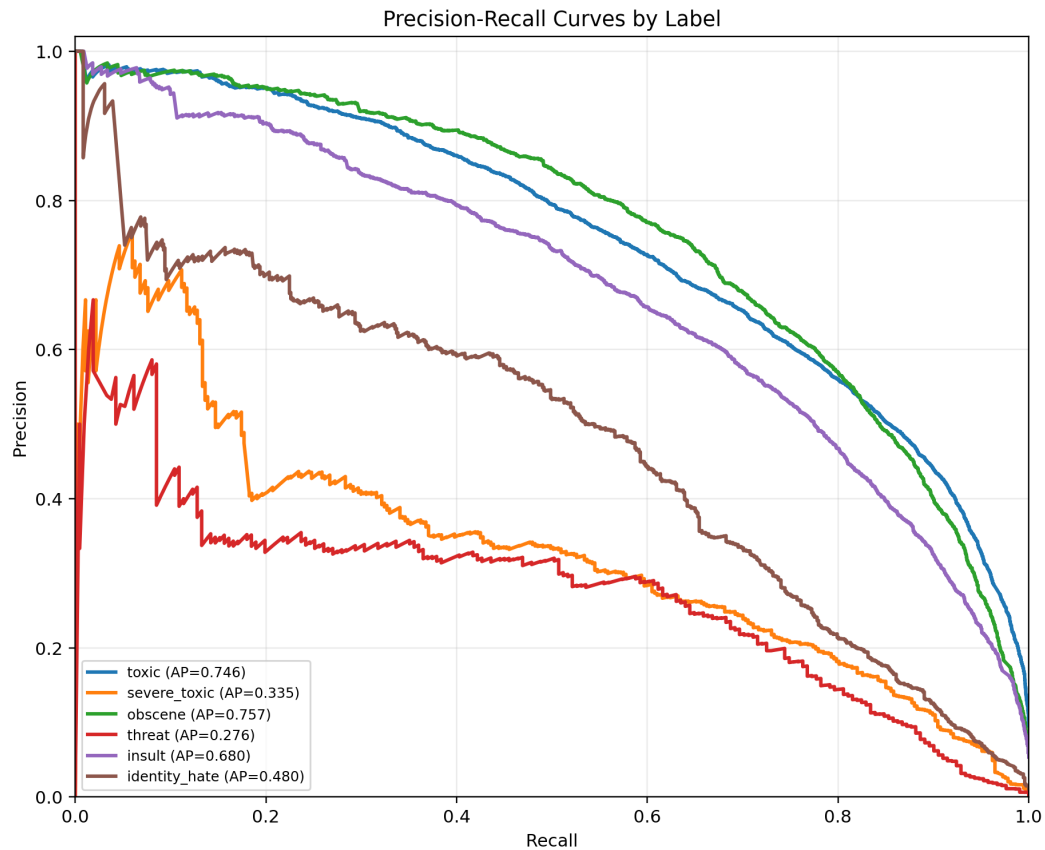


Figure 10: Precision–Recall Curves for Toxicity Labels

The Precision-Recall curves (Figure 10) provide a much more transparent and realistic assessment of the model’s predictive power on this heavily skewed dataset. By plotting Precision against Recall, these curves entirely ignore true negatives, directly exposing how false positives degrade the reliability of positive predictions. The PR curves clearly stratify the model’s performance based on class frequency.

The more highly represented labels—*obscene*, *toxic*, and *insult*—maintain relatively strong precision even as recall increases, yielding higher Average Precision (AP) scores of 0.757, 0.746, and 0.680, respectively. In stark contrast, the curves for the heavily underrepresented classes—*severe_toxic* (AP=0.335) and *threat* (AP=0.276)—experience a rapid, steep collapse in precision at even modest recall levels. This visual divergence perfectly echoes the earlier F1-score analysis: while the model can successfully rank genuinely toxic content above benign content globally (driving high ROC-AUC), attempting to capture a high percentage of the rarest toxic instances inevitably drowns out the true positives with a surge of false alarms (driving low PR-AUC).

3.11 Summary of Traditional Pipeline

Overall, the Traditional Machine Learning Pipeline combines aggressive lexical normalisation, sparse TF-IDF feature extraction, class-weighted one-vs-rest logistic regression, Optuna-based regularisation tuning, multilabel-stratified validation, and per-label threshold calibration. Although this approach cannot model contextual semantics as effectively as transformer-based embeddings, it is efficient, interpretable, and well suited to high-dimensional sparse text classification. It therefore provides a scientifically meaningful baseline for measuring the empirical value of the Deep Learning Pipeline.

4 Deep Learning Pipeline

4.1 Feature Extraction

For the deep learning pipeline, we transitioned from traditional sparse representations (such as TF-IDF) to dense, contextualized word embeddings. To achieve this, we utilized the **DistilRoBERTa** model via the `sentence-transformers` library.

DistilRoBERTa is a compact version of the standard RoBERTa model that employs a student-teacher knowledge distillation paradigm. It was specifically selected because it provides a highly efficient balance between computational cost and predictive performance. By training a 6-layer encoder-only transformer (the student) to emulate a larger 12-layer RoBERTa model (the teacher), DistilRoBERTa reduces the total parameter count from 125 million to roughly 82 million—an approximate 35% decrease in size. Despite halving the number of encoder layers, the architecture preserves the teacher’s 768 hidden dimensionality, 12 self-attention heads, and maximum sequence capacity of 512 tokens. This allows the model to achieve a $2\times$ inference speedup compared to standard RoBERTa, while still preserving the vast majority of its representational power and natural language understanding capabilities.

We implemented the pre-trained `all-distilroberta-v1` model to encode our pre-processed and cleaned text data. Unlike traditional word-level tokenizers, this specific sentence-transformer instance is fine-tuned to map sentences and paragraphs into a semantically meaningful dense vector space. It achieves this by passing the text through the transformer layers and applying a mean pooling operation over the resulting token embeddings. This mechanism maps each entire comment into a fixed-size dense vector. Specifically, for every input sentence, the model produces a single NumPy array of 768 dimensions ($\mathbf{x} \in \mathbb{R}^{768}$).

The embedding process was applied to both the training and testing sets:

- **Training Embeddings:** The cleaned training comments were encoded into a feature matrix of shape $N_{\text{train}} \times 768$.
- **Testing Embeddings:** The cleaned testing comments were similarly encoded into a feature matrix of shape $N_{\text{test}} \times 768$.

To optimize the training workflow and prevent redundant computations in future iterations, the resulting high-dimensional feature matrices were saved directly to disk as binary NumPy files (`X_train_embeddings.npy` and `X_test_embeddings.npy`). These dense feature representations then served as the direct inputs for the downstream deep neural network classifier.

4.2 Model Training

In this phase, we construct and train a neural network to perform multi-label classification on the dense embeddings extracted by the RoBERTa model. The training process is specifically designed to handle high-dimensional representations and the extremely imbalanced nature of our dataset.

4.2.1 Model Architecture

Since the RoBERTa model has already extracted high-quality, context-aware semantic representations, we employ a straightforward linear classifier (a single fully connected layer) rather than a deep, complex architecture. Let N be the batch size. The input to the model is the embedding matrix $X \in \mathbb{R}^{N \times 768}$. The model computes the unnormalized scores (logits) $Z \in \mathbb{R}^{N \times 6}$ for the 6 toxicity labels using the following linear transformation:

$$Z = XW + b$$

where $W \in \mathbb{R}^{768 \times 6}$ represents the learnable weight matrix and $b \in \mathbb{R}^6$ is the bias vector.

4.2.2 Loss Function and Imbalanced Data Handling

Toxicity detection is a multi-label classification problem, meaning a single comment can possess multiple labels simultaneously (e.g., both *toxic* and *insult*). Therefore, we apply the Binary Cross-Entropy (BCE) loss independently to each label.

However, our exploratory data analysis revealed a severe class imbalance (e.g., the *threat* label accounts for less than 1% of the dataset). To prevent the model from aggressively predicting the majority class (clean comments) to minimize errors, we integrate **Smoothed Class Weights** into the loss function. For each label c , the positive weight $w_{pos,c}$ is calculated using the square root of the inverse class ratio:

$$w_{pos,c} = \sqrt{\frac{N_{neg,c}}{N_{pos,c}}}$$

Using the square root acts as a smoothing factor. It provides a sufficient penalty for misclassifying rare labels without over-distorting the model's overall distribution. The weighted BCE loss for a single sample and a specific label c is defined as:

$$L_c = -[w_{pos,c} \cdot y_c \log(\sigma(z_c)) + (1 - y_c) \log(1 - \sigma(z_c))]$$

where $y_c \in \{0, 1\}$ is the ground-truth label and $\sigma(z_c)$ is the sigmoid activation function that converts the logit z_c into a probability. The total loss for a batch is the mean of the losses across all samples and labels.

4.2.3 Optimization Strategy

To minimize the loss function efficiently, we utilize the **Adam** (Adaptive Moment Estimation) optimizer. The hyperparameters and training strategies are configured as follows:

- **Learning Rate:** Set to 0.01. Since the architecture consists of a single linear layer, a relatively large learning rate combined with Adam’s adaptive nature ensures rapid convergence.
- **Batch Size:** The data is processed in mini-batches of 1024 samples. This size is optimally chosen to provide stable gradient estimates while effectively utilizing the GPU’s VRAM bandwidth.
- **Epochs:** The model is trained for 30 epochs, providing sufficient iterations for the weight matrix W to establish the optimal linear decision boundaries within the 768-dimensional space.

4.3 Model Evaluation

After the neural network is fully trained, we evaluate its performance on the unseen test set. The dense embeddings of the test comments are fed into the network, and the raw output logits are passed through a Sigmoid activation function to obtain the predicted probabilities $P(y_c = 1|x)$ for each label. To align with the challenge’s evaluation guidelines, test samples with un-scored labels (marked as -1) are explicitly filtered out prior to metric computation.

4.3.1 Thresholding and Metrics Selection

To convert the continuous predicted probabilities into discrete binary multi-label predictions, a standard global threshold of 0.5 is applied. Due to the extreme class imbalance in toxicity detection, relying solely on Subset Accuracy (Exact Match Ratio) can be highly misleading. Therefore, our evaluation prioritizes the following robust metrics:

- **Macro ROC-AUC:** Evaluates the model’s ability to distinguish between positive and negative classes across all labels equally, regardless of their support size.
- **Macro F1-Score:** The harmonic mean of Precision and Recall, calculated independently for each label and then averaged, heavily penalizing models that ignore rare classes like *threat* or *identity_hate*.

- **Hamming Loss:** Measures the fraction of the total discrete labels that are incorrectly predicted.
- **Per-label Metrics:** Granular computation of Precision, Recall, and F1-score for each of the six toxicity classes.

4.3.2 Standardized Reporting Architecture

To ensure a rigorous and objective comparison between the Traditional Machine Learning pipeline and this Deep Learning approach, we implemented a unified evaluation module. The evaluation script autonomously computes all metrics and exports them into standardized, machine-readable formats.

Specifically, the comprehensive evaluation results are generated into an `evaluation_output_dl/` directory containing:

1. `metrics_summary.json`: Contains overarching metrics such as Macro ROC-AUC, Micro/Macro F1, and Subset Accuracy.
2. `classification_report.json`: Stores the detailed Scikit-Learn classification report.
3. `per_label_metrics.csv`: A tabular format detailing the Support, Precision, Recall, F1, and individual ROC-AUC scores for each specific toxicity label.

This synchronized reporting methodology guarantees that the Deep Learning and Traditional pipelines are evaluated under identical frameworks, allowing for seamless downstream analysis and visualization.

4.4 Deep Learning Pipeline Evaluation Results

Following the extraction of dense contextual embeddings via DistilRoBERTa and the training of the PyTorch neural network, the Deep Learning Pipeline was evaluated on the official test set consisting of 63,978 valid aligned samples. To ensure a rigorous and fair comparison with the Traditional Machine Learning Pipeline, we discarded the default 0.5 decision boundary and performed per-label threshold calibration on the predicted probabilities to maximize the F1-score for each category.

The optimal thresholds applied for the final deep learning binary predictions were:

$$\begin{aligned}\tau_{toxic} &= 0.75, & \tau_{severe_toxic} &= 0.85, & \tau_{obscene} &= 0.75, \\ \tau_{threat} &= 0.80, & \tau_{insult} &= 0.70, & \tau_{identity_hate} &= 0.80.\end{aligned}$$

4.4.1 Aggregate Evaluation Results

Utilizing these optimized thresholds, the model achieves a **micro F1-score of 0.62** and a **macro F1-score of 0.55**. While the micro F1-score is identical to that of the Traditional Pipeline (0.62), the macro F1-score shows a substantial improvement, rising from 0.526 to 0.550.

Because the macro F1-score weights all six classes equally, this increase mathematically indicates that the Deep Learning model is significantly more capable of identifying minority toxicity classes without sacrificing performance on majority classes. The dense contextual embeddings successfully capture abusive semantics that the sparse lexical baseline was unable to represent. Furthermore, the model maintains a high **macro ROC-AUC of 0.9704**, slightly surpassing the traditional baseline.

4.4.2 Per-Label Performance

Analyzing the per-label metrics reveals the empirical advantage of DistilRoBERTa embeddings over TF-IDF. The traditional pipeline struggled heavily with the rarest categories, whereas the deep learning model shows marked improvements. For instance, the F1-score for the highly elusive *threat* label jumped significantly to **0.47** (up from 0.37), and *severe_toxic* improved to **0.37** (up from 0.34).

A key observation is the significantly improved balance between precision and recall. The traditional model achieved high recall at the expense of precision, leading to many false positives. In contrast, the deep learning model tightens this gap; for the *toxic* label, precision increased to 0.62 (from 0.53). This suggests that deep semantic understanding allows the model to better differentiate between genuinely toxic intent and benign comments that happen to contain "flagged" keywords.

Category	Precision	Recall	F1	ROC-AUC	PR-AUC (AP)	Support
toxic	0.62	0.67	0.65	0.9472	0.7156	6090
severe_toxic	0.29	0.52	0.37	0.9823	0.3080	367
obscene	0.66	0.64	0.65	0.9605	0.7185	3691
threat	0.40	0.55	0.47	0.9885	0.4661	211
insult	0.61	0.64	0.63	0.9620	0.6809	3427
identity_hate	0.46	0.60	0.52	0.9818	0.5040	712

Table 10: Per-label classification performance for the Deep Learning Pipeline after threshold optimization.

4.4.3 Receiver Operating Characteristic (ROC) and Precision-Recall Analysis

To evaluate the threshold-independent ranking capability, we analyze the ROC and Precision-Recall (PR) curves illustrated in Figure 11.

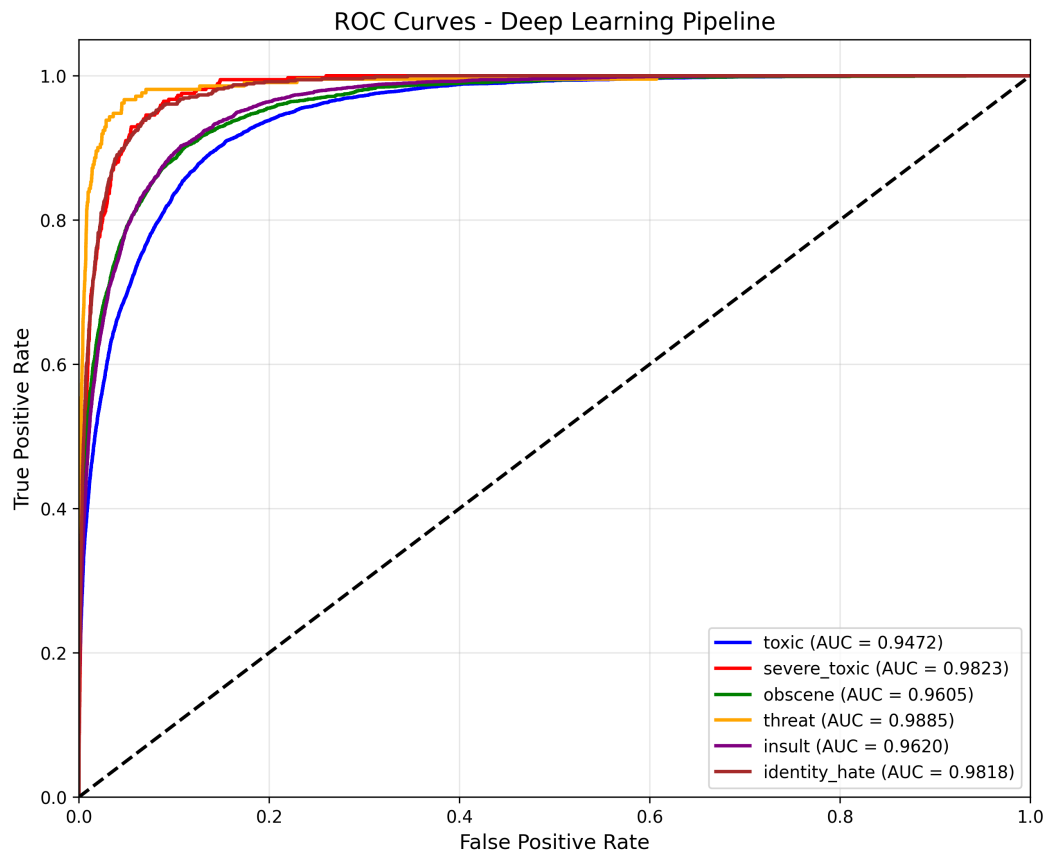


Figure 11: ROC Curves

As shown in the ROC Curves, the Deep Learning pipeline maintains exceptional separability, with all AUC scores exceeding 0.94. The Precision-Recall Curves provide a more realistic assessment on this skewed dataset. The model demonstrates phenomenal gains in the most challenging categories: most notably, the Average Precision (AP) for *threat* skyrocketed to **0.4661** (compared to 0.2762 in the traditional model). This proves that semantic sentence embeddings provide a much more robust feature space for isolating rare toxic behaviors.

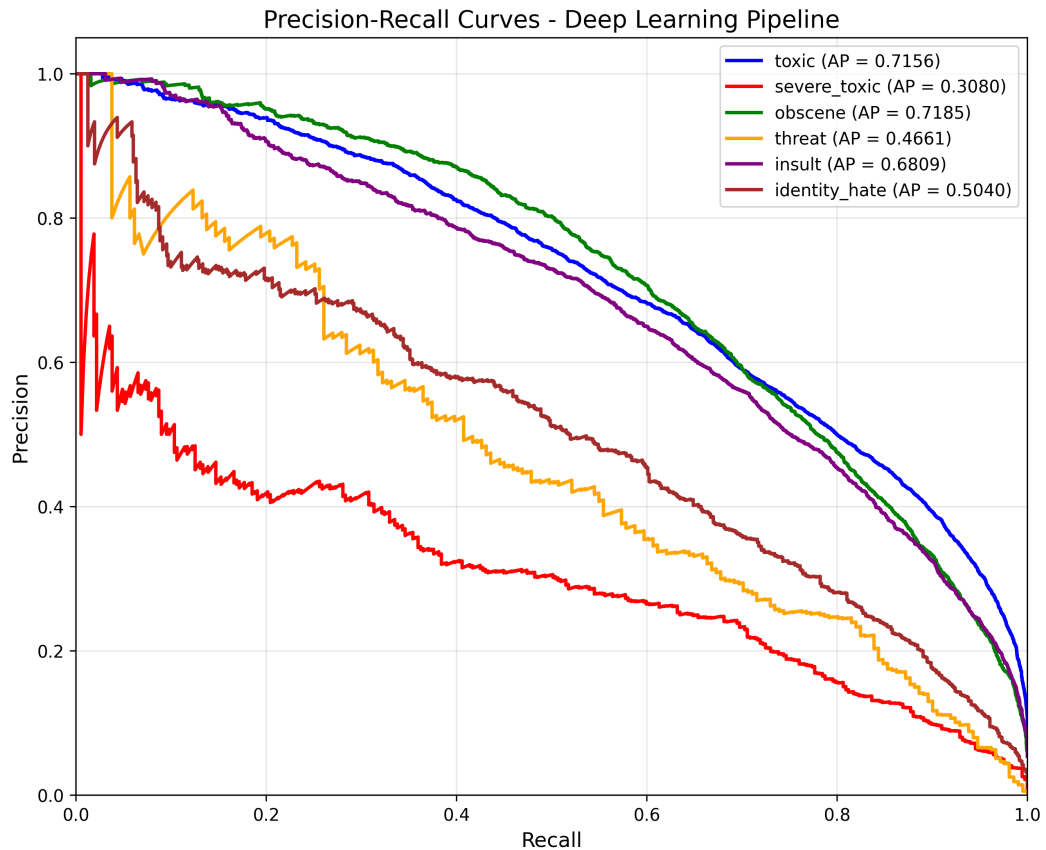


Figure 12: Precision-Recall Curves

4.4.4 Summary of Deep Learning Pipeline

In conclusion, the Deep Learning Pipeline successfully leverages DistilRoBERTa's contextual embeddings to outperform the traditional baseline. By replacing surface-level word counts with deep semantic representations, and coupling them with smoothed class weights and per-label threshold optimization, the model improves classification precision across all categories. The most significant achievement is the enhanced ability to detect rare forms of toxicity (*threat* and *identity_hate*) while maintaining competitive global performance.